Sprint 4 · Reto «Herramientas internas»

Ficha completa del reto, para consultar. Para trabajar con ella abierta, usad el reto en una página.

- Volver al README del sprint · README del sprint en PDF · ecofriendly
- **Esta ficha en PDF:** a color · ecofriendly, para imprimir
- Si es vuestro primer reto: Cómo se trabaja un reto.

RETO Ficha del reto

1 Qué os pide el cliente

Qué os pide el cliente. El equipo de reservas de TurisData calcula cada semana sus cifras **a mano en una hoja de cálculo**: es lento y cada persona lo hace distinto. Os pide una **herramienta interna**, un programa en Python que lea el fichero de reservas, calcule **tres KPI** (cifras que resumen cómo va el negocio: la ocupación, el ingreso medio por reserva y la tasa de cancelación por canal), **avise con claridad si el fichero está mal** y guarde el resultado en un fichero JSON que otros sistemas puedan leer.

Cómo se trabaja. No hace falta experiencia previa: os damos una **plantilla casi hecha** (funciones vacías y pruebas que os dicen si lo que escribís es correcto) y vuestro trabajo es rellenarla paso a paso. La ocupación será solo un *proxy* (una medida aproximada), porque el fichero no trae la capacidad real del hotel. En clase tenéis 10 minutos de planning (día 1) y **100 minutos de reto + 15 de entrega** (día 2); lo que no quepa va a casa o a la 6.ª hora. El equipo decide hasta dónde llega: mínimo, estándar o extra.

2 · Recursos del reto

Recurso	Para qué lo usaréis	Dónde está
Python 3.11 (solo biblioteca estándar: `csv`, `json`, `argparse`, `datetime`)	Escribir y ejecutar `kpis.py`	Ya instalado en el aula; si no, el de Colab
Editor y terminal: VS Code (recomendado) o Colab	Editar el código y ejecutar `python test_kpis.py`	En Colab, subid la carpeta y poned `!` delante: `!python test_kpis.py`
La plantilla `reto_herramientas/`	`kpis.py` con funciones vacías, `test_kpis.py`, `README_PLANTILLA.md` y `plantilla_kpis_pandas.py` (extra)	En GitHub: *Code* → *Download ZIP*. Trabajad en una copia
El CSV real `reservas_turisdata.csv`	6000 reservas **sintéticas**	Botón *Download raw file*; dejadlo **junto a `kpis.py`**
Cuadernos del sprint: `S04_03` y `S04_04`	Las pistas para las funciones (diccionarios, CSV, JSON, excepciones)	Colab; su código explicado
Git y GitHub + cuaderno `S04_05`	Guardar versiones (*commits*) y entregar	Terminal o VS Code. Si falla, subid los ficheros desde la web (*Add file* → *Upload files*): cada subida es un commit
Cómo entregar en GitHub	Subir la carpeta `sprint-04/`	Guía de entrega
Autoevaluación y coevaluación	Cada persona rellena su hoja	Hoja del sprint

Recurso	Para qué lo usaréis	Dónde está
Un asistente de IA (opcional)	Entender errores y pedir pistas	Con las reglas de la «Ayuda 1» del paso a paso

3 · Qué entregar

Este apartado responde a **dos preguntas distintas**, una tabla para cada una:

Tabla A · Los ficheros: qué ficheros hay, de dónde sale cada uno (ya hecho, lo modificáis, lo creáis o lo genera el programa) y **cuáles se suben y adónde**.

Tabla B · Los requisitos: qué tiene que cumplir lo que entregáis y hasta dónde llegar (mínimo, estándar, extra). **Es lo que se evalúa.**

En total, al final del sprint se suben a `sprint-04/` : 4 ficheros del equipo (`kpis.py` , `test_kpis.py` , `README.md` , `salida.json`) **+ 1 autoevaluación y coevaluación por persona**, es decir, **4 + tantos ficheros como personas haya en el equipo** (un equipo de 3 sube 7). «El reto» no es un fichero aparte: son esos cuatro ficheros. Además, el CSV va a `data/` si aún no está, y los extras solo si los hacéis. La *review* y la *retro* se hacen de palabra: no se suben.

Tabla A · Los ficheros: de dónde sale cada uno y cuáles se suben

Todo lo que se sube va al repositorio privado del equipo (`ifcd107-equipo-…`), en la carpeta **`sprint-04/`**, salvo el CSV.

Fichero	Origen	Qué hacéis vosotros	¿Se sube?
`kpis.py`	**Lo modificáis.** Os damos una plantilla con las funciones vacías (`calcular_kpis` y la estructura de `main` ya vienen hechas)	Escribís el código de las funciones sobre la propia plantilla	**Sí** · `sprint-04/`
`test_kpis.py`	**Ya hecho.** Pruebas completas	Nada: se copia tal cual y **no se modifica**	**Sí**, sin cambios · `sprint-04/`
`README.md`	**Lo creáis a partir de una plantilla.** Os damos `README_PLANTILLA.md` con huecos	Copiarla como `README.md` y rellenarla	**Sí** · `sprint-04/`
`salida.json`	**Lo genera el programa** al ejecutarlo con el CSV real	Ejecutar el programa; no se escribe a mano	**Sí** · `sprint-04/`
`reservas_turisdata.csv`	**Ya hecho.** Los datos reales (sintéticos)	Nada: se copia junto a `kpis.py` sin cambiarlo	**Sí**, sin cambios · `data/` (si aún no está)
Autoevaluación y coevaluación (`autoevaluacion_nombre.md`)	**Ya hecha, en blanco**	Cada persona copia la plantilla con su nombre y la rellena; la coevaluación va **dentro del mismo fichero**	**Sí**, **una por persona** (no una por equipo) · `sprint-04/`
`kpis_pandas.py` (extra)	**Lo modificáis.** Parte de `plantilla_kpis_pandas.py`	Solo si el equipo llega al nivel extra	Solo si lo hacéis · `sprint-04/`
Pruebas propias (extra)	**Lo creáis vosotros**, en un fichero aparte	Solo si el equipo llega al nivel extra	Solo si lo hacéis · `sprint-04/`
Copias de prueba del paso 4 (`sin_col.csv` …)	**Las creáis** para romper el programa a propósito	Usarlas y borrarlas	**No se suben**

Tabla B · Los requisitos: qué tiene que cumplir y hasta dónde llegar

Requisito	Qué se pide exactamente	Nivel	Dónde se comprueba
Leer el CSV	`leer_reservas`: convierte los tipos y **descarta y cuenta** las filas defectuosas (noches < 1, precio ≤ 0, `cancelada` distinto de 0/1, datos que no se pueden convertir)	Mínimo	`kpis.py`
Tasa de cancelación por canal	canceladas ÷ reservas del canal × 100, con 1 decimal	Mínimo	`kpis.py` · `tasa_cancelacion_por_canal`
Ingreso medio por reserva	media de `noches × precio_noche` de las reservas **no canceladas**, con 2 decimales (`0.0` si no hay)	Mínimo	`kpis.py` · `ingreso_medio_por_reserva`
Ocupación (proxy)	noches no canceladas ÷ (días del periodo × habitaciones) × 100; el periodo va de la primera a la última llegada. En el mínimo, con ayuda del docente	Estándar	`kpis.py` · `ocupacion_proxy`
Guardar el resultado	`exportar_json` y `main`: JSON en UTF-8 con los tres KPI y la sección `calidad` (filas válidas, descartadas y avisos). En el mínimo, con ayuda del docente	Estándar	`kpis.py` → `salida.json`
Avisar de los errores	Fichero inexistente o columna que falta → mensaje claro y código de salida 1, sin `Traceback`	Mínimo	`kpis.py` · `main` y `leer_reservas`
Resultado con el CSV real	`python kpis.py reservas_turisdata.csv salida.json --capacidad 40`	Mínimo	`salida.json`
Pruebas	`python test_kpis.py` pasa todo. **No se modifican** las dadas; las vuestras, aparte	Estándar	`test_kpis.py`
Código limpio	Docstring en cada función y estilo PEP 8	Estándar	`kpis.py`
README	Cómo se ejecuta y se prueba · qué es cada KPI · por qué la ocupación es un proxy · uso de la IA (qué pedisteis y qué revisasteis) · la revisión con la rúbrica. En el mínimo, versión corta	Mínimo	`README.md`
Git	Al menos 3 *commits* con mensaje claro, de personas distintas, sin claves ni datos personales	Mínimo	Historial del repositorio del equipo
Revisión y autoevaluación	Alguien que **no escribió esa parte** revisa el código y el README con la rúbrica (justo debajo de estas tablas) y anota nivel y prueba en el README; cada persona sube su hoja de autoevaluación y coevaluación	Estándar	`README.md` (sección «Revisión con la rúbrica») y `sprint-04/`
Extras	Pruebas propias · `kpis_pandas.py` con los mismos resultados · argumento `--por-mes` · `@dataclass` para `Reserva`	Extra	`test_kpis.py` · `kpis_pandas.py` · `kpis.py`

Rúbrica del reto · cómo se puntúa y cómo se revisa

La usan **dos veces**: el equipo, para revisar su propio trabajo antes de entregar (es lo que dice la *Definition of Done*), y el docente, para puntuar. Se mira **cada criterio por separado** y se pone el nivel de la columna cuyo texto **se cumple entero**: si lo de un nivel se cumple a medias, es el nivel de abajo. **0** significa que no se ha entregado nada de ese criterio.

Orientación: lo que pide el nivel mínimo sale en un **2**, el estándar en un **3** y los extras en un **4**.

Criterio	1 · Empezado	2 · Mínimo	3 · Estándar	4 · Extra
Corrección técnica (¿funciona?)	Hay código, pero fallan muchas pruebas o no se ejecuta con el CSV real	Pasan las pruebas de lectura, cancelación e ingreso medio; sale `salida.json` con el CSV real; un error de fichero o de columna da mensaje y código 1	`python test_kpis.py` da **12 de 12**; la ocupación y la sección `calidad` salen bien; cada función tiene docstring y el estilo es PEP 8	Además, pruebas propias que pasan (lista vacía, división entre cero…) o `kpis_pandas.py` con los mismos resultados
Reproducibilidad (¿lo puede repetir otra persona?)	Solo funciona en el ordenador de quien lo escribió	El README tiene los comandos, pero faltan datos (carpeta, versión de Python…)	Otra persona obtiene **el mismo JSON y las pruebas en verde** siguiendo solo el README; hay 3 o más *commits* de personas distintas, sin claves ni datos personales	Además, el README indica versión de Python, estructura de carpetas y el comando exacto
Análisis y comunicación al cliente (¿lo entiende quien no programa?)	El README no explica qué es cada KPI	Define los tres KPI, pero no sus límites	Define cada KPI **y explica por qué la ocupación es un proxy** (qué capacidad se supuso), sin jerga	Además, explica cómo cambia la ocupación si cambia la capacidad, o qué conclusión saca el cliente del resultado
IA responsable (¿se sabe qué hizo la IA y qué comprobasteis?)	No dice si se ha usado IA	Dice que se ha usado IA, sin detalle	Dice **qué se pidió, qué se revisó y qué se cambió**, y quien escribe sabe explicar cualquier línea	Además, cuenta un fallo concreto de la IA que detectaron las pruebas, y menciona que los datos son sintéticos y no hay datos personales

Cómo se revisa (unos 10 minutos, antes de subir; lo hace **una persona que no escribió esa parte**):

1. **Ejecutar**, en una carpeta limpia, `python test_kpis.py` y `python kpis.py reservas_turisdata.csv salida.json --capacidad 40`, copiando solo lo que dice el README. Anotar si algo no funciona.
2. **Leer el README como si fuera el cliente**: ¿se entiende qué hace cada KPI y por qué la ocupación es un proxy? Subrayar lo que no se entienda.
3. **Mirar el código**: ¿tiene docstring cada función? ¿Hay algo que no sepáis explicar? Que quien lo escribió lo explique.
4. **Poner un nivel (1 a 4) en cada criterio, con una frase de prueba** (por ejemplo: «3 · 12 de 12 pruebas, pero falta el docstring de `main`»). Sin prueba escrita, el nivel no vale.
5. **Apuntar el resultado** en la sección «Revisión con la rúbrica» del `README.md` y corregir lo más fácil de arreglar. Lo demás se anota como mejora.

Cada persona, además, se puntúa a sí misma en su hoja de autoevaluación y coevaluación con **esta misma rúbrica**.

---

4 · Guía paso a paso

Son cinco pasos, de **100 minutos en total**. Cada uno dice **qué hacéis** (la lista corta), **cómo se hace** con el detalle, **qué deberíais ver** y **cuándo está hecho**. Las ayudas (IA, errores, atascos) están al final. Si el tiempo aprieta, completad el nivel mínimo y dejad lo demás para casa.

Paso 1 · Preparar la carpeta y ver que las pruebas fallan (día 2, 10 minutos)

1. Descargar la plantilla y el CSV y ponerlos **todos en una misma carpeta**.
2. Abrir esa carpeta en el editor y abrir una terminal en ella.

3. Ejecutar `python test_kpis.py` y ver que casi todo falla.
4. Repartir el trabajo del paso 2.

Cómo se hace

- **Descargar.** Opción rápida: en el repositorio del curso pulsad *Code* → *Download ZIP*, descomprimid y copiad a una carpeta nueva (por ejemplo `sprint-04`) estos ficheros: de `sprints/S04/3_Reto/4_Plantillas_y_entrega/reto_herramientas/`, `kpis.py`, `test_kpis.py` y `README_PLANTILLA.md`; y de `datos/`, `reservas_turisdata.csv`. Quedan los cuatro **juntos, en la misma carpeta**.
- **Abrir la terminal.** En VS Code: *File* → *Open Folder…* (elegid `sprint-04`) y *Terminal* → *New Terminal*. En Colab: subid los ficheros al panel izquierdo y poned `!` delante de cada comando.
- **Comprobar Python.** Escribid `python --version`; debe salir 3.11 o superior. Si da error, probad `python3 --version` y usad `python3` en lugar de `python` en todo el reto.
- **Ejecutar las pruebas.** `python test_kpis.py`

Qué deberíais ver. Casi todas las pruebas con `[ERROR]` y el motivo `NotImplementedError: TODO: leer_reservas`; es lo normal, porque las funciones están vacías. Al final: `1 de 12 pruebas superadas` (la de pandas se omite porque es opcional).

Reparto del paso 2. Todo el equipo hace junto `leer_reservas` (las demás pruebas dependen de ella, así que se hace en una pantalla compartida). Después, una función por persona:

Persona	Función	Minutos
A	`tasa_cancelacion_por_canal`	7
B	`ingreso_medio_por_reserva`	7
C	`ocupacion_proxy`	11
D (si hay 4)	revisa la función de otra persona y prepara el README	—

Hecho cuando: veis las pruebas fallando y cada persona sabe qué función le toca.

Paso 2 · Rellenar las funciones, de una en una (día 2, 45 minutos)

1. Entre todo el equipo, primero `leer_reservas` (con `_convertir_fila`): las demás pruebas dependen de ella.
2. Después, repartid el resto: una persona por función, y otra la revisa.
3. Para cada función: programadla, ejecutad `python test_kpis.py`, mirad qué prueba falla, corregid y repetid. Es el método de los profesionales: trabajar **guiados por las pruebas**.
4. Nunca cambiéis `test_kpis.py` para que pase: el código es el que se adapta a las pruebas.

Cómo se hace. Cada función tiene su docstring en `kpis.py` con lo que debe hacer. Aquí tenéis, además, un ejemplo, cómo pensarla y qué herramienta de Python usar. **No es el código**: lo escribís vosotros.

2.1 · `_convertir_fila` y `leer_reservas` (20 minutos, todo el equipo)

Recibe y devuelve. `_convertir_fila` recibe una fila con todo en texto, `{"fecha_llegada": "2025-01-01", "noches": "2", "precio_noche": "100.0", "canal": "directo", "cancelada": "0"}`, y la devuelve con los tipos buenos: la fecha como `date`, `noches` como `int`, `precio_noche` como `float` y `cancelada` como `int`. `leer_reservas(ruta)` devuelve **dos cosas**: la lista de reservas y la lista de avisos.

Cómo pensarlo, en este orden (así las pruebas se van poniendo en verde una a una):

Orden	Qué hacéis	Pista de Python	Prueba que pasa a verde
1	Convertir los tipos en `_convertir_fila`	`int()`, `float()` y `date.fromisoformat()` ya lanzan `ValueError` si el texto no sirve	`test_leer_reservas_convierte_tipos`

Orden	Qué hacéis	Pista de Python	Prueba que pasa a verde
2	Validar los valores: además del tipo, el valor debe tener sentido (noches ≥ 1, precio > 0, `cancelada` 0 o 1, canal no vacío)	`raise ValueError("noches debe ser >= 1")`	`test_valores_imposibles…`
3	Leer el fichero en `leer_reservas`; si no existe, lanzar un error que **incluya la ruta**	`open(ruta, newline="", encoding="utf8")`, `csv.DictReader` y `except FileNotFoundError`	`test_fichero_inexistente`
4	Comprobar la cabecera: si falta una columna, un error que **nombre las que faltan**	Comparar `lector.fieldnames` con `COLUMNAS_OBLIGATORIAS`	`test_faltan_columnas`
5	Descartar y avisar: probar cada fila, y si falla, guardar el aviso y seguir	`for n, fila in enumerate(lector, start=2)` (la fila 1 es la cabecera), `try` / `except ValueError` y `f"Fila {n} descartada: {error}"`	`test_filas_defectuosas…`

Ejemplo. La fila 3 del fichero tiene `noches = 0` → aviso `Fila 3 descartada: noches debe ser >= 1`, y esa fila no entra en la lista de reservas. **Pista de cuaderno:** `S04_04` (CSV, JSON y `try` / `except`).

2.2 · `tasa_cancelacion_por_canal` (7 minutos)

Recibe y devuelve. La lista de reservas → un diccionario `{canal: porcentaje}`. Con 3 reservas del canal `web` y 1 cancelada: `{"web": 33.3}`. Sin reservas: `{}`.

Orden	Qué hacéis	Pista de Python
1	Crear dos diccionarios vacíos: reservas totales por canal y canceladas por canal	`total = {}` y `canceladas = {}`
2	Recorrer las reservas: sumar 1 al total de su canal y, si `cancelada` vale 1, sumar 1 a las canceladas	`total[c] = total.get(c, 0) + 1` empieza un contador
3	Por cada canal, calcular el porcentaje con 1 decimal	`round(100 * canceladas[c] / total[c], 1)`

Se comprueba con `test_tasa_cancelacion…` (incluye el redondeo). **Pista de cuaderno:** `S04_03` (diccionarios).

2.3 · `ingreso_medio_por_reserva` (7 minutos)

Recibe y devuelve. La lista de reservas → un número con 2 decimales. Con reservas confirmadas de 2×100 €, 4×50 € y 1×120 €: (200 + 200 + 120) ÷ 3 = **173.33**. Sin ninguna confirmada: `0.0` (sin dividir entre cero).

Orden	Qué hacéis	Pista de Python
1	Hacer una lista con `noches × precio_noche` **solo** de las reservas no canceladas	Comprensión de lista con `if r["cancelada"] == 0`
2	Si la lista está vacía, devolver `0.0`	`if not lista:`
3	Si no, la media con 2 decimales	`round(sum(lista) / len(lista), 2)`

Se comprueba con `test_ingreso_medio…`. **Pista de cuaderno:** `S04_03` (comprensiones de lista).

2.4 · `ocupacion_proxy` (11 minutos)

Recibe y devuelve. La lista de reservas y la capacidad (nº de habitaciones) → un porcentaje con 1 decimal. Ejemplo de la prueba: 7 noches confirmadas, un periodo de 10 días y 5 habitaciones → 7 ÷ (10 × 5) × 100 = **14.0**.

Orden	Qué hacéis	Pista de Python
1	Si la capacidad es ≤ 0, dar un error (no se puede dividir entre cero)	`raise ValueError("La capacidad debe ser mayor que 0")`
2	Si no hay reservas, devolver `0.0`	`if not reservas:`
3	Calcular los días del periodo: de la primera a la última llegada, **contando también el primer día**	`(max(fechas) - min(fechas)).days + 1`
4	Sumar las noches de las reservas **no canceladas**	`sum(...)` sobre una comprensión con `if`
5	Aplicar la fórmula con 1 decimal	`round(100 * noches / (dias * capacidad), 1)`

Se comprueba con `test_ocupacion_proxy`. En el nivel mínimo, esta función se completa con ayuda del docente. Recordad que es un **proxy**: se explica en el README.

Qué deberíais ver al terminar el paso. `11 de 12 pruebas superadas`: solo falla `test_exportar_json_y_main`, que es del paso 3.

Hecho cuando: pasan todas las pruebas salvo la de `exportar_json` y `main`.

Paso 3 · Exportar el resultado y ejecutar el programa entero (día 2, 15 minutos)

1. Programar `exportar_json` y `main`.
2. Ejecutar el programa con el CSV real.
3. Abrir `salida.json` y compararlo con el resultado esperado.

Cómo se hace

- **`exportar_json(datos, ruta)`**: abrid `ruta` para escribir con `encoding="utf8"` y guardad con `json.dump(datos, f, ensure_ascii=False, indent=2)`. Sin `ensure_ascii=False` las tildes saldrían como `\u00f3`. Pista: `S04_04`.
- **`main(argumentos=None)`**: la plantilla ya lee los argumentos (`args.entrada`, `args.salida` y `args.capacidad`). Vosotros, **dentro de un `try`**: (1) `leer_reservas(args.entrada)`; (2) `calcular_kpis(reservas, args.capacidad)`, que ya viene hecha; (3) añadir al resultado la clave `"calidad"` con `filas_validas`, `filas_descartadas` y `avisos` (si hay muchos avisos, guardad solo los 20 primeros); (4) `exportar_json`; (5) imprimir un mensaje de éxito y **devolver 0**. En el `except (FileNotFoundError, ValueError)`: imprimir `ERROR: …` **por `sys.stderr`** y **devolver 1**.
- **Ejecutar:** `python kpis.py reservas_turisdata.csv salida.json --capacidad 40`

Qué deberíais ver. Por pantalla, un mensaje como `OK: 6000 reservas analizadas, 0 descartadas -> salida.json` y, en la carpeta, un fichero `salida.json` con este contenido:

```
{
  "reservas_analizadas": 6000,
  "ocupacion_proxy_pct": 42.0,
  "ingreso_medio_por_reserva": 576.04,
  "tasa_cancelacion_por_canal_pct": {"empresa": 43.3, "directo": 31.5, "web_intermediaria": 53.9, "agencia": 44.0},
  "calidad": {"filas_validas": 6000, "filas_descartadas": 0, "avisos": []}
}
```

Hecho cuando: existe `salida.json` con esas cifras y `python test_kpis.py` da `12 de 12 pruebas superadas` (con `kpis_pandas.py` ausente, la prueba opcional se omite y cuenta como superada). Si las cifras son distintas, mirad la «Ayuda 2».

Paso 4 · Probar que no se rompe (día 2, 15 minutos)

1. Ejecutar el programa con un fichero que no existe.
2. Quitar una columna al CSV (en una copia) y ejecutar de nuevo.
3. Poner un dato incorrecto en una fila (en otra copia) y ejecutar.
4. Comprobar en cada caso que el mensaje es claro, sin el texto rojo de `Traceback`.

Cómo se hace. Trabajad siempre sobre **copias**: el CSV original no se toca.

Caso	Cómo se provoca	Qué deberíais ver	Código de salida
1. Fichero que no existe	`python kpis.py no_existe.csv salida.json`	`ERROR: No se encuentra el fichero: no_existe.csv`	1
2. Falta una columna	Copiad el CSV como `sin_col.csv` y, con el editor, cambiad en la primera línea `precio_noche` por `precio`. Ejecutad con `sin_col.csv`	`ERROR: Faltan columnas obligatorias: ['precio_noche']`	1
3. Dato incorrecto	Copiad el CSV como `con_error.csv` y, con el editor, escribid `abc` como precio de la fila 5 y `0` como noches de la fila 10. Ejecutad con `con_error.csv`	El programa **no se rompe**: `OK: 5998 reservas analizadas, 2 descartadas -> …` y en `calidad.avisos` aparecen `Fila 5 descartada: …` y `Fila 10 descartada: noches debe ser >= 1`	0

El código de salida se mira justo después de ejecutar: en Linux o Mac, `echo $?`; en Windows (cmd), `echo %errorlevel%`; en PowerShell, `echo $LASTEXITCODE`.

Al terminar, borrad las copias de prueba (`sin_col.csv`, `con_error.csv`, el JSON de prueba): **no se suben**.

Hecho cuando: los tres casos se comportan así. Ese será vuestro «provocar un error» en la review.

Paso 5 · README y entrega (día 2, 15 minutos)

1. Copiar `README_PLANTILLA.md` como `README.md` y rellenarlo.
2. Pedir a alguien que no escribió esa parte que lo revise con la rúbrica y anote el resultado en el README.
3. Subir los ficheros a `sprint-04/` en tres o más subidas, hechas por personas distintas.

Cómo se hace

- **El README.** Cada sección de la plantilla pide una cosa. Para el **nivel mínimo** bastan estas cuatro, en pocas líneas:

Sección de la plantilla	Qué escribís
Qué hace	Dos o tres frases: lee el CSV de reservas, calcula tres KPI, avisa si el fichero está mal y guarda un JSON
Cómo se ejecuta y cómo se prueban	Los comandos `python kpis.py reservas_turisdata.csv salida.json --capacidad 40` y `python test_kpis.py`, y el resultado (`12 de 12 pruebas superadas`)
KPIs y decisiones	Qué es cada KPI y **por qué la ocupación es un proxy**. Ejemplo: «No sabemos cuántas habitaciones tiene el hotel: suponemos 40 y, con 20, saldría el doble. Además contamos todas las noches el día de llegada»
Uso de IA y límites	Qué le pedisteis a la IA y qué revisasteis; los datos son sintéticos

Para el **nivel estándar**, completad también «Requisitos», «Gestión de errores» (qué errores controla, qué mensaje da y que devuelve código 1) y la versión con pandas si la hacéis.

- **La revisión con la rúbrica (5 minutos).** Quien revisa sigue los cinco pasos de «Cómo se revisa» (apartado 3) y rellena la sección «Revisión con la rúbrica» del final del README: un nivel de 1 a 4 por criterio y **una frase de prueba**. Si hay tiempo, arreglad lo más fácil; lo demás queda como «lo primero que mejoraríamos». Si sobran minutos, empezad el extra (pruebas propias o `plantilla_kpis_pandas.py`).
- **Subir los ficheros (la vía del curso, desde el navegador).** Seguid Cómo entregar en GitHub: (1) abrid el repositorio del equipo y **entrad primero en la carpeta `sprint-04`**; (2) *Add file* → *Upload files*; (3) arrastrad los ficheros; (4) escribid un mensaje corto y pulsad **Commit changes**. Cada subida es un *commit* con el nombre de quien la hace. Un reparto posible: la persona A sube `kpis.py` («Añado lectura del CSV y KPI»), la B sube `test_kpis.py` y `salida.json` («Añado pruebas y resultado»), la C sube `README.md` («Añado README»). Los nombres de archivo, **en minúsculas y sin espacios**. El CSV va a `data/` del repositorio solo si aún no está.
- **Alternativa con Git en el terminal** (si ya lo habéis practicado en `S04_05`): `git add kpis.py test_kpis.py README.md salida.json`, `git commit -m "Añado lectura del CSV y KPI"` y `git push`. Cada persona lo hace con su propia cuenta.

Qué deberíais ver. En la carpeta `sprint-04` del repositorio, los cuatro ficheros del equipo y una hoja de autoevaluación por persona (**4 + N**); y en el historial (el icono del reloj), al menos tres *commits* con nombres distintos.

Hecho cuando: otra persona podría ejecutar el proyecto siguiendo solo el README, la sección «Revisión con la rúbrica» está rellena y los tres *commits* están en el repositorio.

Después, 15 minutos de entrega y review: cada persona rellena y sube su autoevaluación y coevaluación a `sprint-04/`, y el equipo enseña, en 3–5 minutos, el script con el CSV real, el JSON, un error provocado con su mensaje y las pruebas en verde. Explicad una decisión de diseño, por ejemplo cómo definisteis la ocupación.

Ayuda 1 · Cómo usar la IA en este reto

La IA puede ahorrar tiempo, pero **da código que parece funcionar y no se entiende**. Reglas:

- **Todo lo que dé, lo ejecutáis y lo entendéis.** Si el docente pregunta por una línea, tenéis que saber explicarla.
- **Las pruebas mandan:** si el código de la IA no las pasa, no vale.
- **Declaradlo** en la sección «Uso de IA y límites» del README: qué pedisteis y qué revisasteis.
- **Nada de datos personales ni claves** en una IA.

Para qué	Ejemplo de petición	Qué comprobáis
Entender un error	«Me sale este error: *[mensaje completo]*. Explícame qué significa, sin darme todavía la solución»	Que lo entendéis y lo arregláis vosotros
Entender una función	«Explícame línea por línea qué hace esta función: *[código]*»	Ejecutadla con un ejemplo pequeño
Pedir una pista	«Quiero calcular el % de cancelaciones por canal con la biblioteca estándar. ¿Qué estructura me conviene? No me des el código»	Que respeta «solo biblioteca estándar»
Revisar el estilo	«¿Sigue PEP 8 este código? ¿Qué cambiarías?»	Que las pruebas siguen pasando

Ayuda 2 · Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
`ModuleNotFoundError: kpis`	Se ejecuta `test_kpis.py` desde otra carpeta	Abrir la terminal **dentro** de la carpeta del reto
La media sale rara	Se incluyen reservas canceladas en el ingreso	**Solo las no canceladas**
La ocupación no sale 42.0	Falta el `+ 1` en los días, se cuentan canceladas o la capacidad no es 40	Releer la fórmula y ejecutar con `--capacidad 40`

Error	Por qué ocurre	Cómo evitarlo
Las pruebas fallan todas a la vez	Dependen de `leer_reservas`	Empezar por esa función
El programa se rompe con un fichero mal	No se controlan las excepciones	`try` / `except` con tipos concretos y mensajes claros
Tildes rotas	Falta `encoding="utf8"`	Añadirlo al abrir y escribir ficheros
Cambiar las pruebas para que pasen	Prisa	No tocar las pruebas dadas; añadir las propias aparte
Un solo *commit* gigante	«Lo subo al final»	Commits pequeños mientras avanzáis

Ayuda 3 · Si os atascáis

1. Leed el mensaje de la prueba que falla: dice qué esperaba y qué recibió. Probad la función en una celda con un ejemplo pequeño.
2. Repasad el apartado de esa función en el paso 2 y su cuaderno.
3. Preguntad al equipo y, después, al docente.